

PROJECT DEVELOPMENT PHASE

AI FAQ ASSISTANT

Project Name: AI FAQ ASSISTANT

Team ID: SWTID-2026-9997

Date: 28 September 2026

1. DEVELOPMENT OVERVIEW

AI FAQ Assistant was developed as a web-based FAQ management and intelligent question-answering application using Node.js, Express.js, MongoDB, Mongoose, JWT, bcryptjs, HTML, CSS and JavaScript.

The development follows a modular backend structure with separate models, controllers, routes, middleware and utility layers.

The implementation includes:

- User registration and login
- JWT-based authentication
- Role-based access control
- User management
- FAQ management
- Category management
- AI-based FAQ question matching
- Match confidence calculation
- Related FAQ suggestions
- No-match handling
- Frontend and backend API integration

2. DEVELOPMENT ENVIRONMENT

The project was developed using:

- Node.js
- Express.js
- MongoDB
- Mongoose
- JWT
- bcryptjs
- JavaScript
- HTML5
- CSS

- Visual Studio Code
- REST API

The AI functionality is implemented using a custom JavaScript-based FAQ similarity and matching engine.

3. BACKEND DEVELOPMENT

The backend was developed as a REST API using Node.js and Express.js.

The main application components are:

Models

- User
- FAQ
- Category

Controllers

- Authentication Controller
- User Controller
- FAQ Controller
- Category Controller
- AI Controller

Routes

- Authentication Routes
- User Routes
- FAQ Routes
- Category Routes
- AI Routes

Middleware

- Authentication Middleware
- Authorization / Admin Access
- Error Handling Middleware

Utility

- AI Matching Engine

This structure separates database models, API request handling, business operations, authentication and AI processing.

Show:

- Express initialization
- middleware
- route registration

4. AUTHENTICATION DEVELOPMENT

The authentication module was implemented using **JWT** and **bcryptjs**.

Registration

POST /api/auth/register

The system accepts user details and securely stores the user's password after hashing.

Login

POST /api/auth/login

The system verifies the user's credentials and generates a JWT token after successful authentication.

Protected API Access

The generated JWT token is used to access protected resources.

Authorization: Bearer <JWT_TOKEN>

The authentication middleware verifies the token before allowing access to protected APIs.

```
JS faqController.js JS aiEngine.js JS authController.js X
nan mu-p > src > controllers > JS authController.js > login
1  const User = require('../models/User');
2  const bcrypt = require('bcryptjs');
3  const jwt = require('jsonwebtoken');
4
5  // Helper to generate JWT token
6  const generateToken = (id) => {
7    return jwt.sign(
8      { id },
9      process.env.JWT_SECRET || 'naan_mudhalvan_ai_faq_secret_key_2026_super_secure',
10     { expiresIn: '30d' }
11   );
12 };
13
14 // @desc   Register a new user
15 // @route  POST /api/auth/register
16 // @access Public
17 const register = async (req, res, next) => {
18   try {
19     const { name, email, password, confirmPassword, role } = req.body;
20
21     // Validate inputs
22     if (!name || !email || !password) {
23       return res.status(400).json({
24         success: false,
25         message: 'Please provide name, email, and password'
26       });
27     }
28
29     if (confirmPassword && password !== confirmPassword) {
30       return res.status(400).json({
31         success: false,
32         message: 'Passwords do not match'
33       });
34     }
35
36     if (password.length < 6) {
37       return res.status(400).json({
38         success: false,
39         message: 'Password must be at least 6 characters long'
40       });
41     }
42
43     // Check duplicate email
44     const existingUser = await User.findOne({ email: email.toLowerCase() });
45     if (existingUser) {
46       return res.status(400).json({
47         success: false,
48         message: 'Email address is already registered'
49       });
50     }
51
52     // Hash password securely with bcrypt
53     const salt = await bcrypt.genSalt(10);
54     const hashedPassword = await bcrypt.hash(password, salt);
```

```

JS faqController.js JS aiEngine.js JS authController.js X
nan mu-p > src > controllers > JS authController.js > login
17 const register = async (req, res, next) => {
55
56 // Create user
57 const user = await User.create({
58   name,
59   email: email.toLowerCase(),
60   password: hashedPassword,
61   role: role && ['admin', 'user'].includes(role) ? role : 'user'
62 });
63
64 // Generate Token
65 const token = generateToken(user._id);
66
67 res.status(201).json({
68   success: true,
69   message: 'User registered successfully',
70   token,
71   user: {
72     id: user._id,
73     name: user.name,
74     email: user.email,
75     role: user.role,
76     createdAt: user.createdAt
77   }
78 });
79 } catch (error) {
80   next(error);
81 }
82 };
83
84 // @desc   Authenticate user & get token
85 // @route   POST /api/auth/login
86 // @access  Public
87 const login = async (req, res, next) => {
88   try {
89     const { email, password } = req.body;
90
91     if (!email || !password) {
92       return res.status(400).json({
93         success: false,
94         message: 'Please enter both email and password'
95       });
96     }
97
98     // Check user & select password explicitly
99     const user = await User.findOne({ email: email.toLowerCase() }).select('+password');
100
101     if (!user) {
102       return res.status(401).json({
103         success: false,
104         message: 'Invalid email or password'
105       });
106     }
107     // Verify password with bcrypt
108     const isMatch = await bcrypt.compare(password, user.password);
109
110     if (!isMatch) {
111       return res.status(401).json({
112         success: false,
113         message: 'Invalid email or password'
114       });
115     }
116     // Generate token
117     const token = generateToken(user._id);
118     res.status(200).json({
119       success: true,
120       message: 'Login successful',
121       token,
122       user: {
123         id: user._id,
124         name: user.name,
125         email: user.email,
126         role: user.role,
127         createdAt: user.createdAt
128       }
129     });
130   } catch (error) {
131     next(error);
132   }
133 }
134 // @desc   Get current user profile
135 // @route   GET /api/auth/me
136 // @access  Private
137 const getMe = async (req, res, next) => {
138   try {
139     const user = await User.findById(req.user.id);
140     res.status(200).json({
141       success: true,
142       user: {
143         id: user._id,
144         name: user.name,
145         email: user.email,
146         role: user.role,
147         createdAt: user.createdAt
148       }
149     });
150   } catch (error) {
151     next(error);
152   }
153 };
154 module.exports = {
155   register,
156   login,
157   getMe
158 };

```

Figure 1: User Registration and Login Implementation

```

JS faqController.js JS aiEngine.js JS authController.js JS authMiddleware.js X
nan mu-p > src > middleware > JS authMiddleware.js > protect
1  const jwt = require('jsonwebtoken');
2  const User = require('../models/User');
3
4  // Protect routes - User must be authenticated
5  const protect = async (req, res, next) => {
6    let token;
7
8    if (
9      req.headers.authorization &&
10      req.headers.authorization.startsWith('Bearer')
11    ) {
12      token = req.headers.authorization.split(' ')[1];
13    }
14
15    if (!token) {
16      return res.status(401).json({
17        success: false,
18        message: 'Not authorized to access this route. Token missing.'
19      });
20    }
21
22    try {
23      const decoded = jwt.verify(token, process.env.JWT_SECRET || 'naan_mudhalvan_ai_faq_secret_key_2026_super_secure');
24
25      req.user = await User.findById(decoded.id);
26
27      if (!req.user) {
28        return res.status(401).json({
29          success: false,
30          message: 'User belonging to this token no longer exists.'
31        });
32      }
33      next();
34    } catch (err) {
35      return res.status(401).json({
36        success: false,
37        message: 'Not authorized. Invalid or expired token.'
38      });
39    }
40  };
41
42  // Admin role check middleware
43  const admin = (req, res, next) => {
44    if (req.user && req.user.role === 'admin') {
45      next();
46    } else {
47      return res.status(403).json({
48        success: false,
49        message: 'Access denied: Requires administrator privilege.'
50      });
51    }
52  };
53
54  module.exports = { protect, admin };

```

Figure 2: JWT Authentication Middleware

5. ROLE-BASED ACCESS CONTROL DEVELOPMENT

The application supports two user roles:

- Admin
- User

Role-based authorization is implemented to restrict administrative operations.

Admin can:

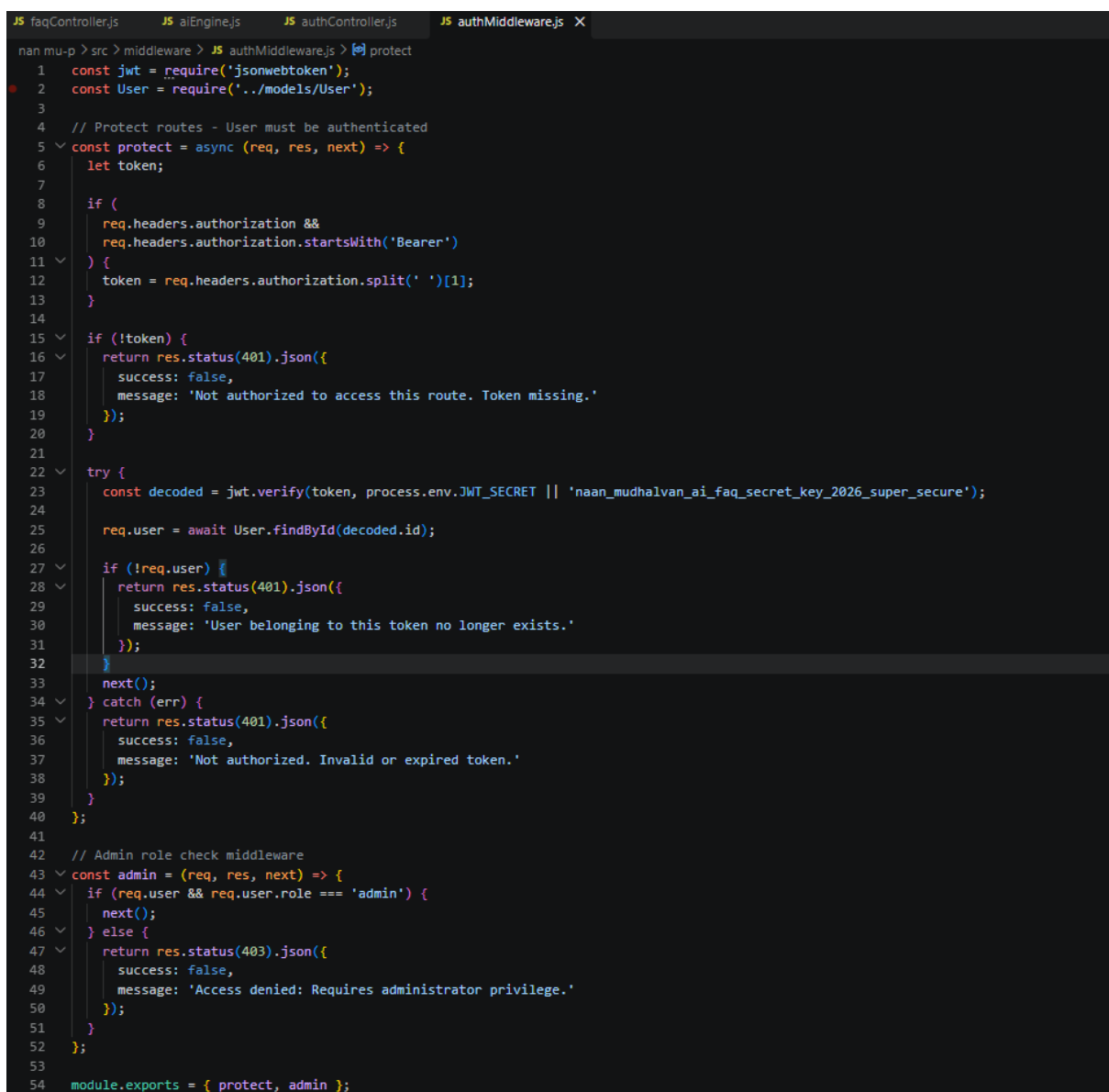
- Manage users

- Create FAQs
- Update FAQs
- Delete FAQs
- Manage categories

User can:

- Access available FAQs
- Use the AI FAQ Assistant
- Search/query FAQ information

Protected API operations verify the authenticated user's role before performing restricted actions.



```

nan mu-p > src > middleware > JS authMiddleware.js > protect
1  const jwt = require('jsonwebtoken');
2  const User = require('../models/User');
3
4  // Protect routes - User must be authenticated
5  const protect = async (req, res, next) => {
6    let token;
7
8    if (
9      req.headers.authorization &&
10      req.headers.authorization.startsWith('Bearer')
11    ) {
12      token = req.headers.authorization.split(' ')[1];
13    }
14
15    if (!token) {
16      return res.status(401).json({
17        success: false,
18        message: 'Not authorized to access this route. Token missing.'
19      });
20    }
21
22    try {
23      const decoded = jwt.verify(token, process.env.JWT_SECRET || 'naan_mudhalvan_ai_faq_secret_key_2026_super_secure');
24
25      req.user = await User.findById(decoded.id);
26
27      if (!req.user) {
28        return res.status(401).json({
29          success: false,
30          message: 'User belonging to this token no longer exists.'
31        });
32      }
33      next();
34    } catch (err) {
35      return res.status(401).json({
36        success: false,
37        message: 'Not authorized. Invalid or expired token.'
38      });
39    }
40  };
41
42  // Admin role check middleware
43  const admin = (req, res, next) => {
44    if (req.user && req.user.role === 'admin') {
45      next();
46    } else {
47      return res.status(403).json({
48        success: false,
49        message: 'Access denied: Requires administrator privilege.'
50      });
51    }
52  };
53
54  module.exports = { protect, admin };

```

Figure 3: Role-Based Access Control

6. FAQ MANAGEMENT DEVELOPMENT

The FAQ module was developed with complete CRUD operations.

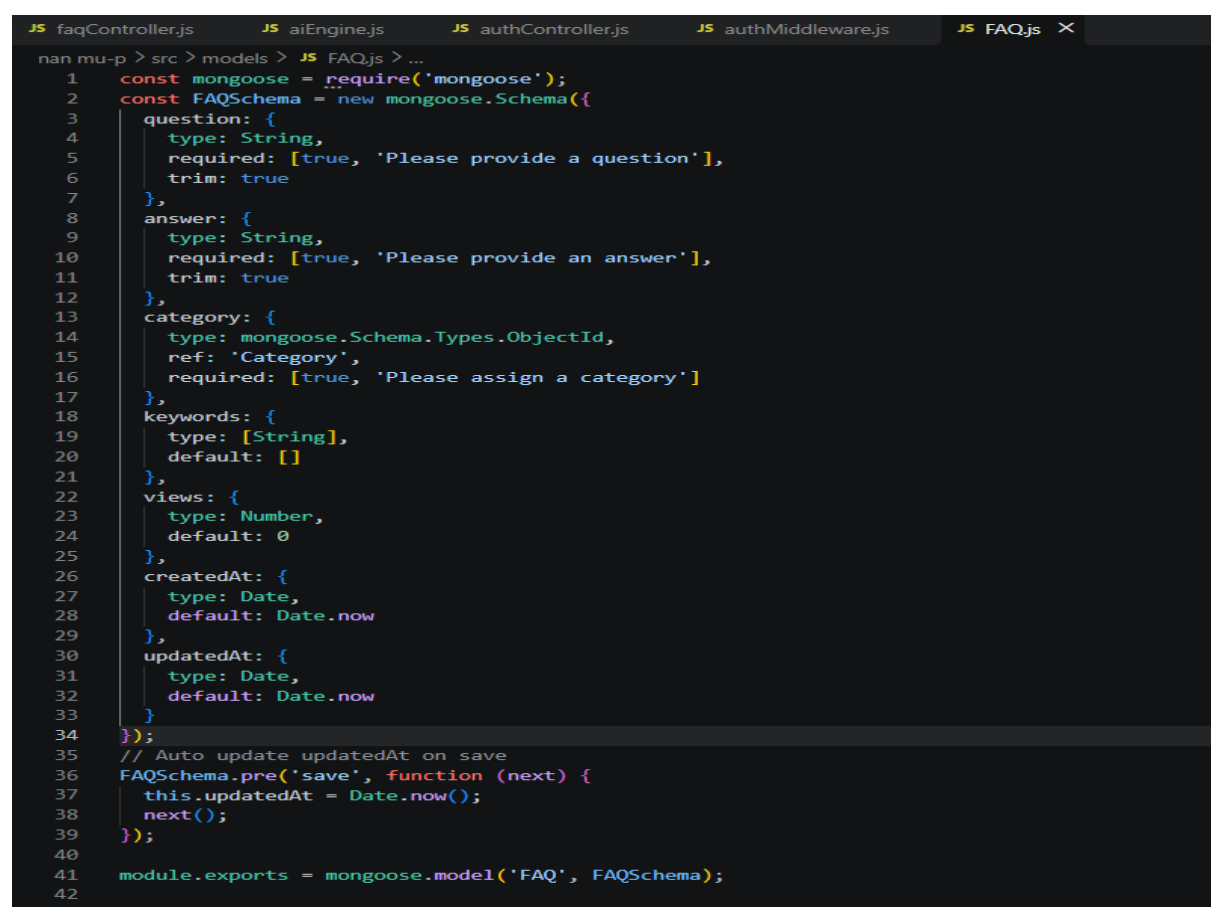
APIs

POST /api/faqs
GET /api/faqs
GET /api/faqs/:id
PUT /api/faqs/:id
DELETE /api/faqs/:id

The FAQ module contains:

- Question
- Answer
- Category
- Keywords
- Views
- Created date
- Updated date

Administrators can create, update and delete FAQ records, while users can retrieve FAQ information.



```
JS faqController.js JS aiEngine.js JS authController.js JS authMiddleware.js JS FAQ.js X
nan mu-p > src > models > JS FAQ.js > ...
1 const mongoose = require('mongoose');
2 const FAQSchema = new mongoose.Schema({
3   question: {
4     type: String,
5     required: [true, 'Please provide a question'],
6     trim: true
7   },
8   answer: {
9     type: String,
10    required: [true, 'Please provide an answer'],
11    trim: true
12  },
13  category: {
14    type: mongoose.Schema.Types.ObjectId,
15    ref: 'Category',
16    required: [true, 'Please assign a category']
17  },
18  keywords: {
19    type: [String],
20    default: []
21  },
22  views: {
23    type: Number,
24    default: 0
25  },
26  createdAt: {
27    type: Date,
28    default: Date.now
29  },
30  updatedAt: {
31    type: Date,
32    default: Date.now
33  }
34 });
35 // Auto update updatedAt on save
36 FAQSchema.pre('save', function (next) {
37   this.updatedAt = Date.now();
38   next();
39 });
40
41 module.exports = mongoose.model('FAQ', FAQSchema);
42
```

Figure 4: FAQ Database Schema

```

JS faqController.js X JS aiEngine.js JS authController.js JS authMiddleware.js JS FAQ.js
nan mu-p > src > controllers > JS faqController.js > createFAQ > faq
1 const FAQ = require('../models/FAQ');
2 const Category = require('../models/Category');
3
4 // @desc Get all FAQs (with optional category & search filter)
5 // @route GET /api/faqs
6 // @access Public
7 const getFAQs = async (req, res, next) => {
8   try {
9     const { category, search } = req.query;
10    let query = {};
11
12    if (category) {
13      query.category = category;
14    }
15
16    if (search && search.trim()) {
17      const searchRegex = new RegExp(search.trim(), 'i');
18      query.$or = [
19        { question: searchRegex },
20        { answer: searchRegex },
21        { keywords: searchRegex }
22      ];
23    }
24
25    const faqs = await FAQ.find(query)
26      .populate('category', 'name description')
27      .sort({ createdAt: -1 });
28
29    res.status(200).json({
30      success: true,
31      count: faqs.length,
32      data: faqs
33    });
34  } catch (error) {
35    next(error);
36  }
37 };
38
39 // @desc Get single FAQ by ID
40 // @route GET /api/faqs/:id
41 // @access Public
42 const getFAQById = async (req, res, next) => {
43   try {
44     const faq = await FAQ.findById(req.params.id).populate('category', 'name description');
45     if (!faq) {
46       return res.status(404).json({
47         success: false,
48         message: 'FAQ not found'
49       });
50     }
51
52     // Increment view count
53     faq.views = (faq.views || 0) + 1;
54     await faq.save();
55
56     res.status(200).json({
57       success: true,
58       data: faq
59     });
60   } catch (error) {
61     next(error);
62   }
63 };
64
65 // @desc Create FAQ
66 // @route POST /api/faqs
JS faqController.js X JS aiEngine.js JS authController.js JS authMiddleware.js JS FAQ.js
nan mu-p > src > controllers > JS faqController.js > createFAQ > faq
67 // @access Private/Admin
68 const createFAQ = async (req, res, next) => {
69   try {
70     const { question, answer, category, keywords } = req.body;
71
72     if (!question || !answer || !category) {
73       return res.status(400).json({
74         success: false,
75         message: 'Question, Answer, and Category are required'
76       });
77     }
78
79     // Verify category exists
80     const categoryExists = await Category.findById(category);
81     if (!categoryExists) {
82       return res.status(404).json({
83         success: false,
84         message: 'Selected Category does not exist'
85       });
86     }
87
88     // Parse keywords array
89     let keywordArray = [];
90     if (Array.isArray(keywords)) {
91       keywordArray = keywords.map(k => k.trim()).filter(Boolean);
92     } else if (typeof keywords === 'string') {
93       keywordArray = keywords.split(',').map(k => k.trim()).filter(Boolean);
94     }
95
96     const faq = await FAQ.create({
97       question: question.trim(),
98       answer: answer.trim(),
99       category,
100       keywords: keywordArray
101     });
102
103     const populatedFaq = await FAQ.findById(faq._id).populate('category', 'name description');
104
105     res.status(201).json({
106       success: true,
107       message: 'FAQ created successfully',
108       data: populatedFaq
109     });
110   } catch (error) {
111     next(error);
112   }
113 };
114
115 // @desc Update FAQ
116 // @route PUT /api/faqs/:id
117 // @access Private/Admin
118 const updateFAQ = async (req, res, next) => {
119   try {
120     const { question, answer, category, keywords } = req.body;
121
122     let faq = await FAQ.findById(req.params.id);
123     if (!faq) {
124       return res.status(404).json({
125         success: false,
126         message: 'FAQ not found'
127       });
128     }
129
130     if (category) {
131       const categoryExists = await Category.findById(category);
132       if (!categoryExists) {
133         return res.status(404).json({

```

```

JS faqController.js X JS aiEngine.js JS authController.js JS authMiddleware.js JS FAQ.js
nan mu-p > src > controllers > JS faqController.js > createFAQ > faq
117 const updateFAQ = async (req, res, next) => {
118     return res.status(404).json({
119         success: false,
120         message: 'Selected Category does not exist'
121     });
122     faq.category = category;
123 }
124 if (question !== undefined) faq.question = question.trim();
125 if (answer !== undefined) faq.answer = answer.trim();
126
127 if (keywords !== undefined) {
128     if (Array.isArray(keywords)) {
129         faq.keywords = keywords.map(k => k.trim()).filter(Boolean);
130     } else if (typeof keywords === 'string') {
131         faq.keywords = keywords.split(',').map(k => k.trim()).filter(Boolean);
132     }
133 }
134 faq.updatedAt = Date.now();
135 await faq.save();
136
137 const updatedFaq = await FAQ.findById(faq._id).populate('category', 'name description');
138
139 res.status(200).json({
140     success: true,
141     message: 'FAQ updated successfully',
142     data: updatedFaq
143 });
144 } catch (error) {
145     next(error);
146 }
147 };
148
149 // @desc Delete FAQ
150 // @route DELETE /api/faqs/:id
151 // @access Private/Admin
152 const deleteFAQ = async (req, res, next) => {
153     try {
154         const faq = await FAQ.findById(req.params.id);
155         if (!faq) {
156             return res.status(404).json({
157                 success: false,
158                 message: 'FAQ not found'
159             });
160         }
161         await faq.deleteOne();
162         res.status(200).json({
163             success: true,
164             message: 'FAQ deleted successfully'
165         });
166     } catch (error) {
167         next(error);
168     }
169 };
170
171 module.exports = {
172     getFAQs,
173     getFAQById,
174     createFAQ,
175     updateFAQ,
176     deleteFAQ
177 };
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195

```

Figure 5: FAQ CRUD Implementation

7. CATEGORY DEVELOPMENT

The Category module was developed to organize FAQ information.

APIs

GET /api/categories

POST /api/categories

GET /api/categories/:id

PUT /api/categories/:id

DELETE /api/categories/:id

Category creation, updating and deletion are protected according to the user's role.

JS faqController.js JS categoryController.js X JS aiEngine.js JS authController.js JS authMiddleware.js JS FAQ.js

```
nan mu-p > src > controllers > JS categoryController.js > ...
1  const Category = require('../models/Category');
2  const FAQ = require('../models/FAQ');
3  // @desc   Get all categories
4  // @route   GET /api/categories
5  // @access  Public
6  const getCategories = async (req, res, next) => {
7    try {
8      const categories = await Category.find().sort({ name: 1 });
9      // Include FAQ count for each category
10     const categoriesWithCount = await Promise.all(
11       categories.map(async (cat) => {
12         const count = await FAQ.countDocuments({ category: cat._id });
13         return {
14           ...cat.toObject(),
15           faqCount: count
16         };
17       })
18     );
19     res.status(200).json({
20       success: true,
21       count: categoriesWithCount.length,
22       data: categoriesWithCount
23     });
24   } catch (error) {
25     next(error);
26   }
27 };
28 // @desc   Get single category by ID
29 // @route   GET /api/categories/:id
30 // @access  Public
31 const getCategoryById = async (req, res, next) => {
32   try {
33     const category = await Category.findById(req.params.id);
34     if (!category) {
35       return res.status(404).json({
36         success: false,
37         message: 'Category not found'
38       });
39     }
40     const faqCount = await FAQ.countDocuments({ category: category._id });
41     res.status(200).json({
42       success: true,
43       data: {
44         ...category.toObject(),
45         faqCount
46       }
47     });
48   } catch (error) {
49     next(error);
50   }
51 };
52 // @desc   Create category
53 // @route   POST /api/categories
54 // @access  Private/Admin
55 const createCategory = async (req, res, next) => {
```

JS faqController.js JS categoryController.js X JS aiEngine.js JS authController.js JS authMiddleware.js JS FAQ.js

```
nan mu-p > src > controllers > JS categoryController.js > ...
55 const createCategory = async (req, res, next) => {
56   try {
57     const { name, description } = req.body;
58     if (!name || !name.trim()) {
59       return res.status(400).json({
60         success: false,
61         message: 'Category name is required'
62       });
63     }
64     const existingCategory = await Category.findOne({ name: name.trim() });
65     if (existingCategory) {
66       return res.status(400).json({
67         success: false,
68         message: 'Category with this name already exists'
69       });
70     }
71     const category = await Category.create({
72       name: name.trim(),
73       description: description ? description.trim() : ''
74     });
75     res.status(201).json({
76       success: true,
77       message: 'Category created successfully',
78       data: category
79     });
80   } catch (error) {
81     next(error);
82   }
83 };
84 // @desc   Update category
85 // @route   PUT /api/categories/:id
86 // @access  Private/Admin
87 const updateCategory = async (req, res, next) => {
88   try {
89     const { name, description } = req.body;
90     let category = await Category.findById(req.params.id);
91     if (!category) {
92       return res.status(404).json({
93         success: false,
94         message: 'Category not found'
95       });
96     }
97     if (name && name.trim() !== category.name) {
98       const existing = await Category.findOne({ name: name.trim() });
99       if (existing) {
100         return res.status(400).json({
101           success: false,
102           message: 'Another category with this name already exists'
103         });
104       }
105       category.name = name.trim();
106     }
107     if (description !== undefined) {
108       category.description = description.trim();
109     }
```

```

JS faqController.js JS categoryController.js X JS aiEngine.js JS authController.js JS authMiddleware.js JS FAQ.js
nan mu-p > src > controllers > JS categoryController.js > ...
87  const updateCategory = async (req, res, next) => {
109  }
110  await category.save();
111  res.status(200).json({
112    success: true,
113    message: 'Category updated successfully',
114    data: category
115  });
116  } catch (error) {
117    next(error);
118  }
119  };
120  // @desc   Delete category
121  // @route  DELETE /api/categories/:id
122  // @access Private/Admin
123  const deleteCategory = async (req, res, next) => {
124    try {
125      const category = await Category.findById(req.params.id);
126      if (!category) {
127        return res.status(404).json({
128          success: false,
129          message: 'Category not found'
130        });
131      }
132      // Check if category has associated FAQs
133      const faqCount = await FAQ.countDocuments({ category: category._id });
134      if (faqCount > 0) {
135        return res.status(400).json({
136          success: false,
137          message: `Cannot delete category containing ${faqCount} active FAQ(s). Please reassign or delete the FAQs first.`
138        });
139      }
140      await category.deleteOne();
141      res.status(200).json({
142        success: true,
143        message: 'Category deleted successfully'
144      });
145    } catch (error) {
146      next(error);
147    }
148  };
149  module.exports = {
150    getCategories,
151    getCategoryById,
152    createCategory,
153    updateCategory,
154    deleteCategory
155  };
156

```

Figure 6: Category Management Implementation

8. AI FAQ ASSISTANT DEVELOPMENT

The AI Assistant is the main intelligent component of the project.

The system accepts a user's question through:
POST /api/ai/ask

The submitted question is processed by the custom AI matching engine.

The system compares the user's question with stored FAQ information and identifies the most relevant FAQ answer.

```
JS faqController.js JS categoryController.js JS aiController.js X JS aiEngine.js JS authController.js JS auth
nan mu-p > src > controllers > JS aiController.js > askAI
1  const FAQ = require('../models/FAQ');
2  const { processAiQuery } = require('../utils/aiEngine');
3
4  // @desc    Ask AI FAQ Assistant
5  // @route    POST /api/ai/ask
6  // @access   Public
7  const askAI = async (req, res, next) => {
8    try {
9      const { question } = req.body;
10     if (!question || !question.trim()) {
11       return res.status(400).json({
12         success: false,
13         message: 'Please provide a valid question for the AI FAQ Assistant'
14       });
15     }
16
17     // Retrieve all active FAQs with categories
18     const faqs = await FAQ.find().populate('category', 'name description');
19
20     // Run custom AI similarity matching algorithm
21     const aiResult = processAiQuery(question.trim(), faqs);
22
23     res.status(200).json({
24       success: true,
25       query: question.trim(),
26       timestamp: new Date().toISOString(),
27       ...aiResult
28     });
29   } catch (error) {
30     next(error);
31   }
32 };
33
34 module.exports = {
35   askAI
36 };
```

Figure 7: AI FAQ Assistant API Implementation

9. AI MATCHING ENGINE DEVELOPMENT

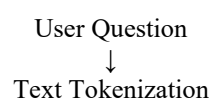
This is the **most important development section** for your project.

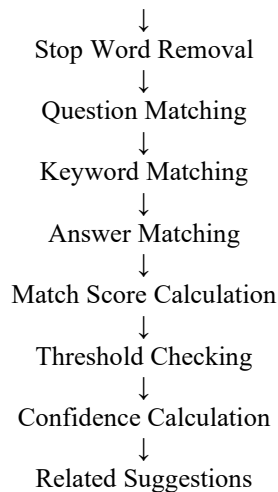
The AI matching engine is implemented in:

src/utils/aiEngine.js

The engine processes the user question through multiple matching stages.

Processing





The engine calculates a matching score using question tokens, keywords and answer-related matches.

A predefined threshold is used to determine whether a suitable FAQ answer is available.

When a relevant FAQ is found, the system returns the answer with confidence information and related suggestions.

If no suitable match is found, the system provides a no-match response with suggested categories.

```

1  const STOP_WORDS = new Set([
2    'a', 'about', 'above', 'after', 'again', 'against', 'all', 'am', 'an', 'and', 'any', 'are', 'aren't',
3    'as', 'at', 'be', 'because', 'been', 'before', 'being', 'below', 'between', 'both', 'but', 'by',
4    'can', 'can't', 'cannot', 'could', 'did', 'do', 'does', 'doing', 'down', 'during', 'each', 'few',
5    'for', 'from', 'further', 'had', 'has', 'have', 'having', 'he', 'her', 'here', 'hers', 'herself',
6    'him', 'himself', 'his', 'how', 'i', 'if', 'in', 'into', 'is', 'it', 'its', 'itself', 'just', 'me',
7    'more', 'most', 'my', 'myself', 'no', 'nor', 'not', 'now', 'of', 'off', 'on', 'once', 'only', 'or',
8    'other', 'our', 'ours', 'ourselves', 'out', 'over', 'own', 'same', 'should', 'so', 'some', 'such',
9    'than', 'that', 'the', 'their', 'theirs', 'there', 'these', 'they', 'this', 'those', 'through', 'to', 'too', 'under',
10   'until', 'up', 'very', 'was', 'we', 'were', 'what',
11   'when', 'where', 'which', 'while', 'who', 'whom', 'why', 'with', 'would', 'you', 'your', 'yours'
12 ]);
13 function tokenize(text) {
14   if (!text) return [];
15   return text
16     .toLowerCase()
17     .replace(/[\s\S]/g, '')
18     .split(/\w/)
19     .filter(word => word.length > 1 && !STOP_WORDS.has(word));
20 }
21
22 function computeSimilarity(userQuery, faq) {
23   const queryTokens = tokenize(userQuery);
24   if (queryTokens.length === 0) return { score: 0, faq };
25   const rawQuery = userQuery.toLowerCase().trim();
26   const rawQuestion = faq.question.toLowerCase().trim();
27   const rawAnswer = faq.answer.toLowerCase().trim();
28   const rawKeywords = Array.isArray(faq.keywords)
29     ? faq.keywords.map(k => k.toLowerCase().trim())
30     : (faq.keywords || '').toLowerCase().split(',').map(k => k.trim());
31
32   let score = 0;
33   // 1. Exact or near-exact question match
34   if (rawQuestion === rawQuery) {
35     score += 150;
36   } else if (rawQuestion.includes(rawQuery) || rawQuery.includes(rawQuestion)) {
37     score += 90;
38   }
39
40   // 2. Keyword exact / token matches
41   rawKeywords.forEach(kw => {
42     if (!kw) return;
43     if (rawQuery.includes(kw)) {
44       score += 40;
45     }
46     const kwTokens = tokenize(kw);
47     kwTokens.forEach(token => {
48       if (queryTokens.includes(token)) {
49         score += 25;
50       }
51     });
52   });
53
54   // 3. Question Token Match
55   const questionTokens = tokenize(faq.question);
56   let matchedTokensCount = 0;
57   queryTokens.forEach(token => {
58     if (questionTokens.includes(token)) {
59       matchedTokensCount++;
60       score += 25;
61     }
62   });
63
64   // 4. Answer Token Match
65   const answerTokens = tokenize(faq.answer);
66   queryTokens.forEach(token => {

```

```

faqController.js  categoryController.js  aiController.js  aiEngine.js X  authController.js  authMiddleware.js  FAQ.js

nan mu-p > src > utils > aiEngine.js > processAiQuery
22 function computeSimilarity(userQuery, faq) {
66   queryTokens.forEach(token => {
67     if (answerTokens.includes(token)) {
68       score += 8;
69     }
70   });
71
72   // Coverage bonus: ratio of query tokens matched in question
73   const coverageRatio = matchedTokensCount / queryTokens.length;
74   score += Math.round(coverageRatio * 30);
75
76   return { score, faq };
77 }
78
79 function processAiQuery(userQuery, faqs) {
80   if (!userQuery || typeof userQuery !== 'string' || !userQuery.trim()) {
81     return {
82       found: false,
83       message: "Please enter a valid question or query."
84     };
85   }
86   if (!faqs || faqs.length === 0) {
87     return {
88       found: false,
89       message: "No FAQ data is currently available in the knowledge base."
90     };
91   }
92   const scoredFaqs = faqs.map(faq => computeSimilarity(userQuery, faq));
93   // Sort descending by score
94   scoredFaqs.sort((a, b) => b.score - a.score);
95   const topMatch = scoredFaqs[0];
96   const threshold = 18; // Minimum threshold required for high-confidence match
97   if (topMatch && topMatch.score >= threshold) {
98     // Get related suggestions (top 2 next matches if score > 18)
99     const related = scoredFaqs
100       .slice(1, 4)
101       .filter(item => item.score >= 12)
102       .map(item => ({
103         id: item.faq_id,
104         question: item.faq.question,
105         category: item.faq.category
106       }));
107     // Confidence calculation (capped at 98%)
108     const confidence = Math.min(98, Math.max(50, Math.round(topMatch.score * 0.8)));
109     return {
110       found: true,
111       faq: topMatch.faq,
112       confidence: `${confidence}%`,
113       related: related,
114       matchScore: topMatch.score
115     };
116   }
117   return {
118     found: false,
119     message: "Sorry, I couldn't find an exact or relevant FAQ answer matching your question in the system database. Please rephrase your query or select a category to browse related topics.",
120     suggestedCategories: Array.from(new Set(faqs.map(f => f.category?.name).filter(Boolean))).slice(0, 5)
121   };
122 }
123 module.exports = {
124   tokenize,
125   computeSimilarity,
126   processAiQuery
127 };
128

```

Figure 8: AI FAQ Matching Engine Implementation

10. AI RESPONSE AND SUGGESTION DEVELOPMENT

The AI response system provides:

- Matched FAQ answer
- Confidence score
- Related FAQ suggestions
- Match score
- No-match response when appropriate

This allows the system to provide useful responses even when the user's question is not an exact match to the stored FAQ.

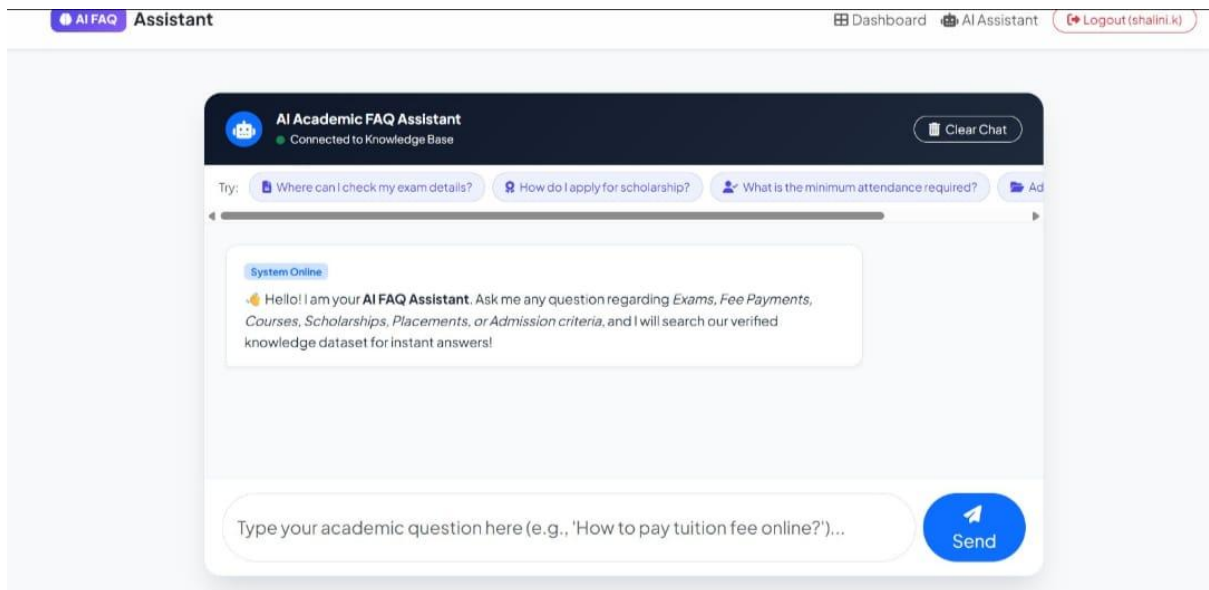
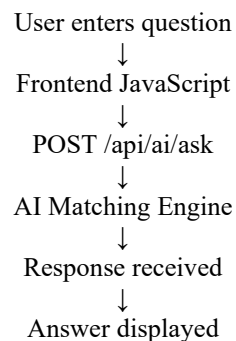


Figure 9: AI FAQ Assistant User Interface

11. FRONTEND AI INTEGRATION

The frontend AI module connects the user interface with the backend AI API.

The processing flow is:



```

JS faqController.js JS categoryController.js JS aiController.js JS aiEngine.js aijs X JS authController.js JS authMiddleware.js JS FAQ.js
nan mu-p > public > js > JS aijs > ...
1 // AI Chatbot Assistant Interface Logic
2
3 document.addEventListener('DOMContentLoaded', () => {
4   const chatForm = document.getElementById('chat-form');
5   const chatInput = document.getElementById('chat-input');
6   const clearChatBtn = document.getElementById('clear-chat-btn');
7
8   if (chatForm) {
9     chatForm.addEventListener('submit', (e) => {
10      e.preventDefault();
11      const userText = chatInput.value.trim();
12      if (userText) {
13        handleUserQuery(userText);
14        chatInput.value = '';
15      }
16    });
17   }
18
19   if (clearChatBtn) {
20     clearChatBtn.addEventListener('click', () => {
21       clearChatHistory();
22     });
23   }
24 });
25
26 function sendQuickPrompt(promptText) {
27   const chatInput = document.getElementById('chat-input');
28   if (chatInput) {
29     chatInput.value = promptText;
30   }
31   handleUserQuery(promptText);
32 }
33
34 async function handleUserQuery(questionText) {
35   const messagesContainer = document.getElementById('chat-messages');
36   if (!messagesContainer) return;
37
38   // 1. Append User Bubble
39   appendMessageBubble('user', questionText);
40   scrollToBottom(messagesContainer);
41
42   // 2. Show Typing Indicator Bubble
43   const typingId = 'typing-' + Date.now();
44   appendTypingIndicator(typingId);
45   scrollToBottom(messagesContainer);
46
47   try {
48     const res = await apiCall('/ai/ask', {
49       method: 'POST',
50       body: { question: questionText }
51     });
52
53     // Remove Typing Indicator
54     removeTypingIndicator(typingId);
55
56     if (res.found) {
57       let assistantMsg = `<div>${res.faq.answer}</div>`;
58
59       // Related questions if any
60       if (res.related && res.related.length > 0) {
61         assistantMsg += `<div class="mt-3 pt-2 border-top small text-muted"><strong>Related FAQs:</strong><ul class="mb-0 mt-1 ps-3">`;
62         res.related.forEach((rel) => {
63           assistantMsg += `<li><a href="#" onclick="sendQuickPrompt('${escapeHtml(rel.question)}');` return false;" class="text-primary text-decoration-none">${rel.question}</a></li>`;
64         });
65         assistantMsg += `</ul></div>`;
66       }
67
68       appendMessageBubble('assistant', assistantMsg, {
69         category: res.faq.category ? res.faq.category.name : 'Matched FAQ',
70         confidence: res.confidence
71       });
72     } else {
73       appendMessageBubble('assistant', res.message, {
74         category: 'Answer Not Found',
75         confidence: null
76       });
77     }
78
79     scrollToBottom(messagesContainer);
80   } catch (err) {
81     removeTypingIndicator(typingId);
82     appendMessageBubble('assistant', `⚠ Error communicating with AI FAQ server: ${err.message}`, {
83       category: 'System Error',
84       confidence: null
85     });
86     scrollToBottom(messagesContainer);
87   }
88 }
89
90 function appendMessageBubble(sender, content, meta = {}) {
91   const messagesContainer = document.getElementById('chat-messages');
92   if (!messagesContainer) return;
93
94   const bubbleDiv = document.createElement('div');
95   const isUser = sender === 'user';
96   bubbleDiv.className = `chat-bubble ${isUser ? 'chat-bubble-user' : 'chat-bubble-assistant'}`;
97
98   let badgeHtml = '';
99   if (isUser && meta.category) {
100    const isErrorOrNotFound = meta.category.includes('Not Found') || meta.category.includes('Error');
101    const badgeColor = isErrorOrNotFound ? 'bg-danger text-white' : 'bg-primary-subtle text-primary';
102    const confHtml = meta.confidence ? `<span class="ms-2 opacity-75"><i class="fas fa-check-circle text-success me-1"></i>Match Confidence: ${meta.confidence}</span>` : '';
103    badgeHtml = `<div class="mb-2"><span class="badge ${badgeColor}">${meta.category}</span> ${confHtml}</div>`;
104   }
105
106   bubbleDiv.innerHTML = `
107     ${badgeHtml}
108     <div>${content}</div>

```

```

109     <div class="text-end mt-1" style="font-size: 0.7rem; opacity: 0.7;">
110         ${new Date().toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' })}
111     </div>
112 `;
113
114     messagesContainer.appendChild(bubbleDiv);
115 }
116
117 function appendTypingIndicator(id) {
118     const messagesContainer = document.getElementById('chat-messages');
119     if (!messagesContainer) return;
120
121     const typingDiv = document.createElement('div');
122     typingDiv.id = id;
123     typingDiv.className = 'chat-bubble chat-bubble-assistant text-muted py-3';
124     typingDiv.innerHTML = `
125         <div class="d-flex align-items-center gap-2">
126             <div class="spinner-grow spinner-grow-sm text-primary" role="status"></div>
127             <div class="spinner-grow spinner-grow-sm text-primary" style="animation-delay: 0.2s" role="status"></div>
128             <div class="spinner-grow spinner-grow-sm text-primary" style="animation-delay: 0.4s" role="status"></div>
129             <span class="ms-2 small">AI Assistant is analyzing FAQ database...</span>
130         </div>
131     `;
132     messagesContainer.appendChild(typingDiv);
133 }
134
135 function removeTypingIndicator(id) {
136     const elem = document.getElementById(id);
137     if (elem) elem.remove();
138 }
139
140 function clearChatHistory() {
141     const messagesContainer = document.getElementById('chat-messages');
142     if (!messagesContainer) return;
143
144     messagesContainer.innerHTML = `
145         <div class="chat-bubble chat-bubble-assistant">
146             <div class="mb-2"><span class="badge bg-success-subtle text-success">System Reset</span></div>
147             Hello! I am your <strong>AI FAQ Assistant</strong>. I am trained on your institution's FAQ dataset. How can I assist you today?
148         </div>
149     `;
150 }
151
152 function scrollToBottom(elem) {
153     elem.scrollTop = elem.scrollHeight;
154 }
155
156 function escapeHtml(str) {
157     return str.replace(/'/g, "\\'").replace(/"/g, '"');
158 }
159

```

Figure 10: Frontend AI API Integration

12. SECURITY DEVELOPMENT

The security layer was implemented using:

- bcryptjs password hashing
- JWT authentication
- Role-based authorization
- Protected routes
- Error handling
- Sensitive password protection

The User model also prevents the password field from being returned in normal JSON responses.

```

JS faqController.js JS categoryController.js JS aiController.js JS aiEngine.js ai.js JS authController.js JS authMiddleware.js X JS FAQ.js
nan mu-p > src > middleware > JS authMiddleware.js > protect
1  const jwt = require('jsonwebtoken');
2  const User = require('../models/User');
3
4  // Protect routes - User must be authenticated
5  const protect = async (req, res, next) => {
6    let token;
7
8    if (
9      req.headers.authorization &&
10      req.headers.authorization.startsWith('Bearer')
11    ) {
12      token = req.headers.authorization.split(' ')[1];
13    }
14
15    if (!token) {
16      return res.status(401).json({
17        success: false,
18        message: 'Not authorized to access this route. Token missing.'
19      });
20    }
21
22    try {
23      const decoded = jwt.verify(token, process.env.JWT_SECRET || 'naan_mudhalvan_ai_faq_secret_key_2026_super_secure');
24
25      req.user = await User.findById(decoded.id);
26
27      if (!req.user) {
28        return res.status(401).json({
29          success: false,
30          message: 'User belonging to this token no longer exists.'
31        });
32      }
33      next();
34    } catch (err) {
35      return res.status(401).json({
36        success: false,
37        message: 'Not authorized. Invalid or expired token.'
38      });
39    }
40  };
41
42  // Admin role check middleware
43  const admin = (req, res, next) => {
44    if (req.user && req.user.role === 'admin') {
45      next();
46    } else {
47      return res.status(403).json({
48        success: false,
49        message: 'Access denied: Requires administrator privilege.'
50      });
51    }
52  };
53
54  module.exports = { protect, admin };

```

Figure 11: Security Implementation

13. API TESTING DURING DEVELOPMENT

Each major API module can be tested during development using an API testing tool such as **Postman**.

Testing covers:

- Registration
- Login
- JWT authentication
- User management
- FAQ APIs

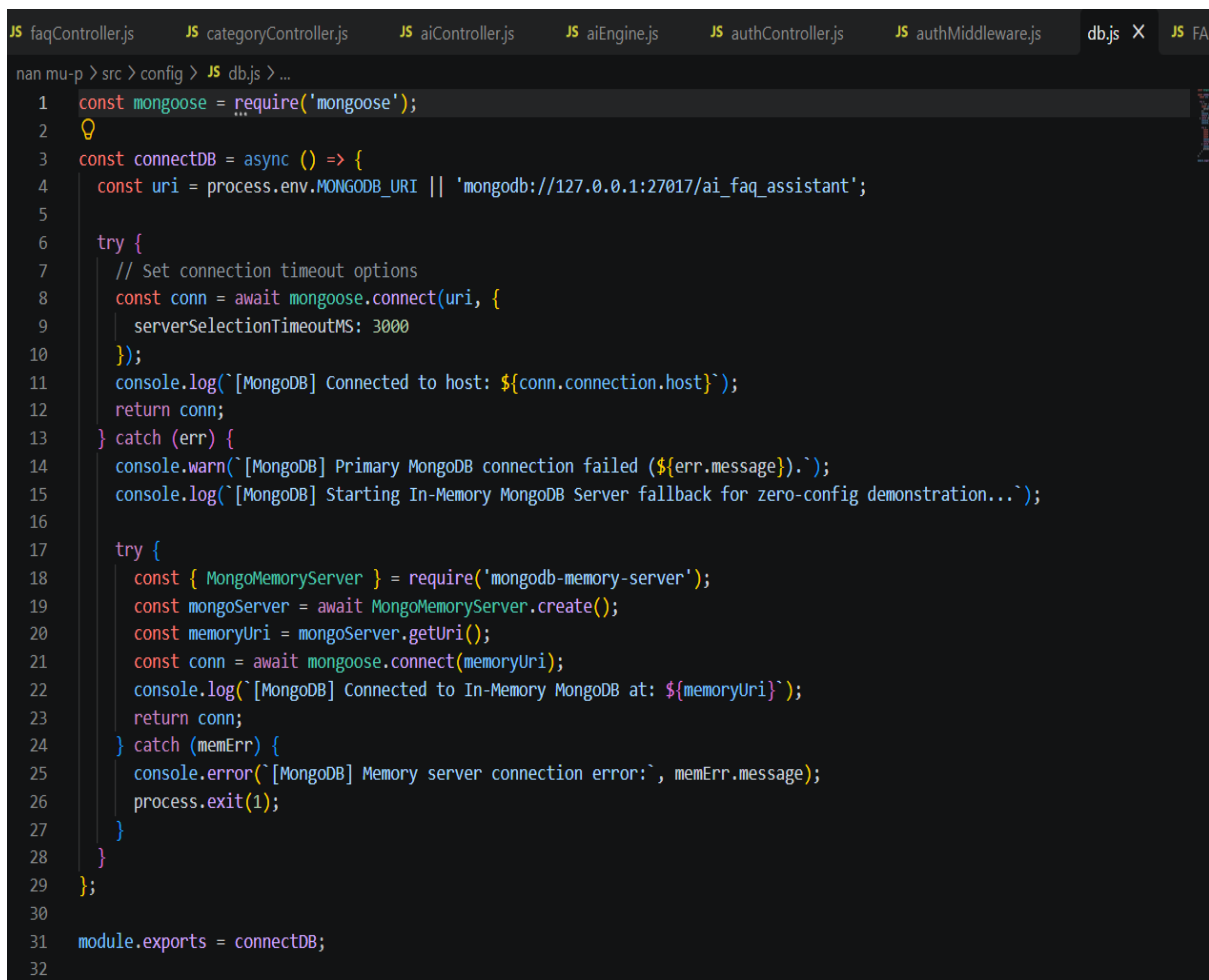
- Category APIs
- AI /api/ai/ask
- Protected admin APIs

14. MONGODB CONNECTION

MongoDB is used as the database for storing:

- Users
- FAQs
- Categories

Mongoose is used to define schemas and communicate with MongoDB.



```
1  const mongoose = require('mongoose');
2
3  const connectDB = async () => {
4    const uri = process.env.MONGODB_URI || 'mongodb://127.0.0.1:27017/ai_faq_assistant';
5
6    try {
7      // Set connection timeout options
8      const conn = await mongoose.connect(uri, {
9        serverSelectionTimeoutMS: 3000
10     });
11     console.log(`[MongoDB] Connected to host: ${conn.connection.host}`);
12     return conn;
13   } catch (err) {
14     console.warn(`[MongoDB] Primary MongoDB connection failed (${err.message}).`);
15     console.log(`[MongoDB] Starting In-Memory MongoDB Server fallback for zero-config demonstration...`);
16
17     try {
18       const { MongoMemoryServer } = require('mongodb-memory-server');
19       const mongoServer = await MongoMemoryServer.create();
20       const memoryUri = mongoServer.getUri();
21       const conn = await mongoose.connect(memoryUri);
22       console.log(`[MongoDB] Connected to In-Memory MongoDB at: ${memoryUri}`);
23       return conn;
24     } catch (memErr) {
25       console.error(`[MongoDB] Memory server connection error:`, memErr.message);
26       process.exit(1);
27     }
28   }
29 };
30
31 module.exports = connectDB;
32
```

Figure 12: MongoDB Database Connection

15. FILE STRUCTURE

The project follows a modular structure.

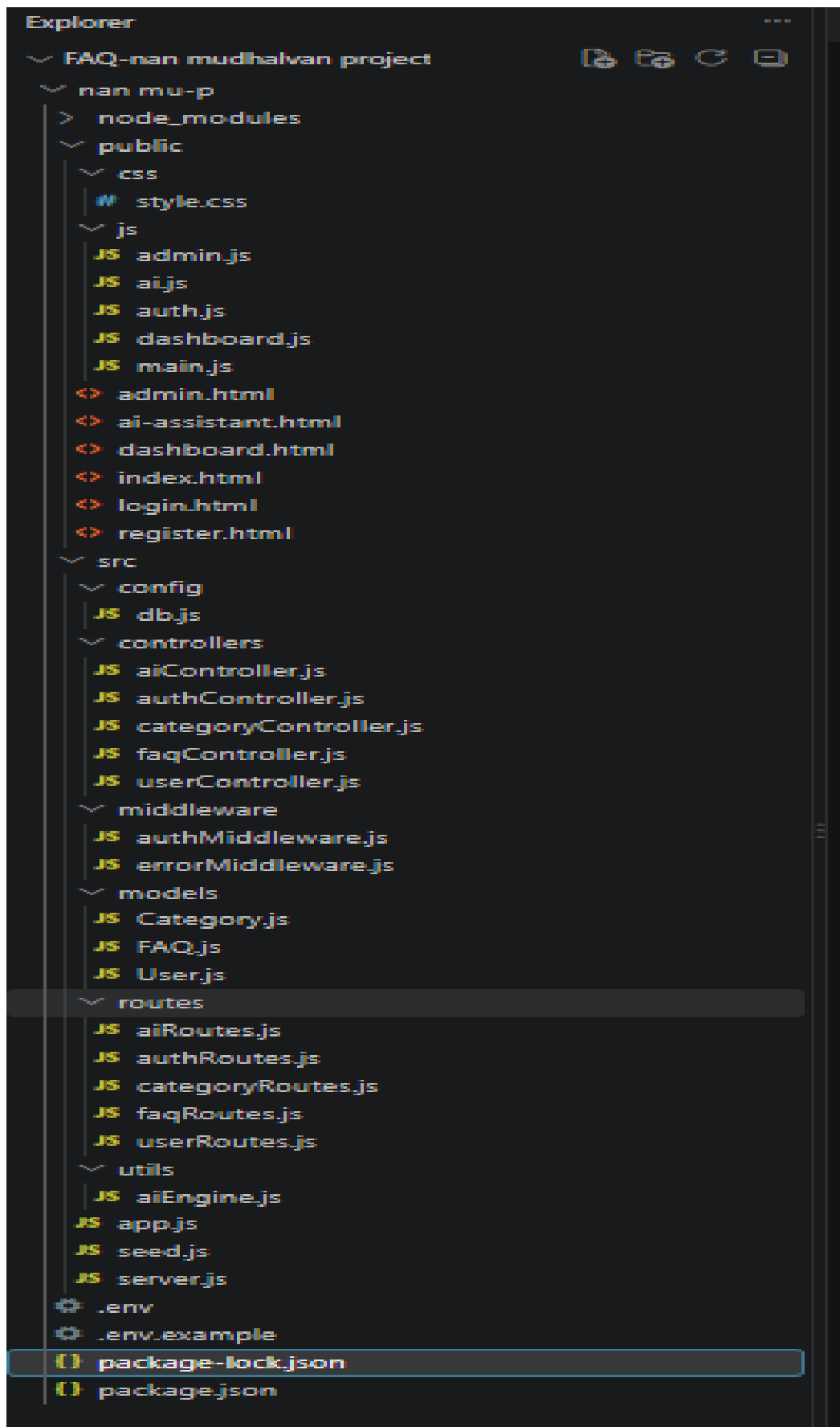


Figure 13: Complete Project File Structure

16. DEVELOPMENT OUTCOME

The development phase resulted in a functional **AI FAQ Assistant** containing:

- Secure user authentication
- JWT-based authorization
- Admin/User role management
- FAQ CRUD operations
- Category management
- MongoDB database integration
- AI FAQ question matching
- Match score calculation
- Confidence information
- Related FAQ suggestions
- No-match handling
- Frontend-backend API integration
- Security middleware

The application follows a modular architecture with separate models, controllers, routes, middleware and AI utility components.

CONCLUSION

The Project Development Phase transformed the requirements and design of the AI FAQ Assistant into an implemented web application. The system integrates Node.js, Express.js, MongoDB, Mongoose, JWT, bcryptjs and a custom JavaScript-based FAQ matching engine. The developed modules include authentication, role-based access, FAQ management, category management and AI-powered FAQ question matching. The frontend and backend components are integrated through REST APIs, providing the foundation for the subsequent testing phase.